BIG DATA ANALYTICS

Apache Pig

Data-flow scripting on Hadoop: features, anatomy, Pig Latin, data types, execution modes and HDFS commands, with a worked example

MOTIVATION

Why Pig? The same job, two ways

~60

lines of Java for Word Count in MapReduce (mapper, reducer, driver)

5

```
lines  = LOAD 'input.txt' AS (line:chararray);
words  = FOREACH lines GENERATE
           FLATTEN(TOKENIZE(line)) AS word;
grp    = GROUP words BY word;
counts = FOREACH grp GENERATE group, COUNT(words);
DUMP counts;
```

Pig is a high-level platform: you describe WHAT happens to the data step by step; Pig writes the MapReduce jobs for you.

WHAT PIG GIVES YOU

Features of Pig

1 Ease of programming

Complex tasks become a short sequence of simple steps. No Java needed.

2 Rich operator set

Built-in LOAD, FILTER, JOIN, GROUP, ORDER, UNION, SPLIT and more.

3 Optimization

Pig rewrites your plan automatically, so you focus on logic, not tuning.

4 Extensibility (UDFs)

Write your own functions in Java, Python, JavaScript or Ruby.

5 Handles any data

Structured, semi-structured and unstructured data; schema optional.

6 Nested data model

Tuples, bags and maps let data stay nested instead of flattened.

TWO MAIN PARTS

Anatomy of Pig

1 Pig Latin: the language

A data-flow language. Each statement takes a relation as input and produces a new relation as output.

2 Pig runtime: the execution environment

Grunt shell or Pig server that parses, optimizes and compiles Pig Latin into jobs, then submits them to Hadoop.

a **Grunt shell**
Interactive command line to type Pig Latin

b **Parser**
Checks syntax and types; builds a logical plan (DAG)

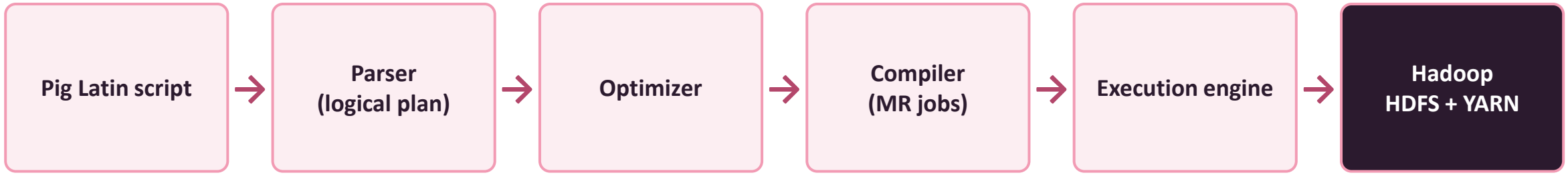
c **Optimizer**
Applies rules such as pushing filters early

d **Compiler**
Turns the optimized plan into MapReduce jobs

e **Execution engine**
Submits jobs to Hadoop in the right order

HOW A SCRIPT BECOMES CLUSTER WORK

Pig on Hadoop



Runs on the client

Pig is installed only on the machine you work from. It talks to the cluster like any Hadoop client.

Lazy until output

Statements only build the plan. DUMP or STORE triggers the whole flow to run.

Data stays in HDFS

Inputs are read from HDFS, and intermediate and final results are written back to HDFS.

WHY IT'S CALLED PIG

Pig Philosophy: four mottos

1 Pigs eat anything

Works with any data: relational, nested, unstructured; with or without a schema.

2 Pigs live anywhere

Not tied to one engine. Runs on MapReduce, Tez or Spark.

3 Pigs are domestic animals

Easy to control and extend: UDFs, custom load/store functions, and you can turn off optimizations.

4 Pigs fly

Processes data quickly; designers focus on performance improvements.

THE LANGUAGE

Pig Latin overview

- **Data flow, step by step:** each statement transforms one relation into a new one.
- **Statements end with a semicolon** and are assigned to names: alias = OPERATOR ...;
- **Case sensitivity:** keywords (LOAD, FILTER) are not case-sensitive; relation names, field names and function names (COUNT, SUM) are.
- **Comments:** -- single line, and /* ... */ for multiple lines.
- **Lazy execution:** nothing runs until DUMP or STORE.
- **Schema optional:** fields can be named and typed with AS, or referred to by position $0, $1, ...

```
-- load students
A = LOAD 'students'
    USING PigStorage(',')
    AS (name:chararray,
        dept:chararray,
        marks:int);

B = FILTER A BY marks > 60;
C = GROUP B BY dept;
D = FOREACH C GENERATE
      group, AVG(B.marks);
DUMP D;  /* runs now */
```

A USEFUL COMPARISON

Pig Latin vs SQL

Aspect	Pig Latin	SQL
Style	Procedural data flow, step by step	Declarative, one statement for the result
Schema	Optional; can be defined at load time	Required, defined before loading
Data model	Nested: tuples, bags, maps	Flat tables
Intermediate results	Named and reusable (each alias)	Hidden inside subqueries
Best for	ETL pipelines, raw/semi-structured data	Querying structured data

Rule of thumb: Pig is for building data pipelines; SQL (or Hive) is for asking questions of clean tables.

Data types: simple (scalar)

Type	Meaning	Example
`int`	32-bit signed integer	10
`long`	64-bit signed integer	10L
`float`	32-bit floating point	10.5F
`double`	64-bit floating point	10.5
`chararray`	String (UTF-8 characters)	'Chennai'
`bytearray`	Raw bytes; default when no type is given	(untyped field)
`boolean`	true / false	true
`datetime`	Date and time	2026-09-25T10:00:00
`biginteger / bigdecimal`	Arbitrary-precision numbers	123456789012345

! Remember

If you LOAD without an AS clause, every field is a bytearray. Pig casts it later when you use it.

NULL means the value is unknown or missing, same idea as in SQL.

DATA TYPES 2 OF 2

Data types: complex

1 Tuple

An ordered set of fields, like one row.

```
(Ravi, CSE, 78)
```

2 Bag

A collection of tuples. Unordered, duplicates allowed.

```
{(Ravi,CSE,78),
(Priya,IT,85)}
```

3 Map

A set of key#value pairs. Keys are chararray.

```
[dept#CSE, year#3]
```

A relation is the outermost bag. So a Pig dataset = a bag of tuples, and tuples can contain bags and maps inside them.

Tiffin-box analogy: tuple = one tiffin box, bag = a big bag of tiffin boxes, map = a labelled locker.

Pig keywords: relational operators

Operator	What it does	Example
`LOAD`	Reads data from the file system	`A = LOAD 'f' USING PigStorage(',');`
`FILTER`	Keeps rows that match a condition	`B = FILTER A BY qty >= 2;`
`FOREACH ... GENERATE`	Selects or computes columns per row	`C = FOREACH A GENERATE id, qty*price;`
`GROUP / COGROUP`	Groups rows by key (one or more relations)	`D = GROUP A BY category;`
`JOIN`	Joins relations on a key	`E = JOIN A BY cid, B BY cid;`
`ORDER BY`	Sorts a relation	`F = ORDER D BY revenue DESC;`
`LIMIT`	Keeps the first N rows	`G = LIMIT F 3;`
`DISTINCT`	Removes duplicate rows	`H = DISTINCT A;`
`UNION / SPLIT`	Combines relations / splits one into many	`SPLIT A INTO x IF qty>2, y IF qty<=2;`
`FLATTEN`	Un-nests a bag or tuple into rows	`FLATTEN(TOKENIZE(line))`

Pig keywords: diagnostics, output and functions

Keyword	Purpose
`DUMP`	Print a relation to the screen (triggers execution)
`STORE ... INTO`	Save a relation to HDFS (triggers execution)
`DESCRIBE`	Show the schema of a relation
`EXPLAIN`	Show logical, physical and MapReduce plans
`ILLUSTRATE`	Run on a small sample and show each step's output

Function	Purpose
`COUNT, SUM, AVG`	Aggregate over a bag
`MAX, MIN`	Largest / smallest in a bag
`TOKENIZE`	Split a string into a bag of words
`CONCAT, SIZE`	Join strings / size of a value
`PigStorage(',')`	Load/store function with a delimiter

Built-in function names are case-sensitive: COUNT works, count does not.

Running Pig: three ways

1 Grunt shell (interactive)

Type Pig Latin line by line and see results immediately. Best for learning and exploring.

```
$ pig -x local
grunt> A = LOAD 'f.csv';
grunt> DUMP A;
```

2 Script (batch) mode

Save statements in a .pig file and run it. Best for repeatable jobs; supports -param.

```
$ pig myscript.pig
$ pig -param IN=/data
      myscript.pig
```

3 Embedded mode

Run Pig from a Java program (PigServer) or extend it with UDFs.

```
PigServer ps =
  new PigServer("mapreduce");
ps.registerQuery(...);
```

WHERE THE WORK RUNS

Execution modes of Pig

L Local mode

Runs in a single JVM on your machine and uses the local file system. Fast to start; for testing with small data.

Command: pig -x local

MR MapReduce mode (default)

Runs on the Hadoop cluster and reads/writes HDFS. For real, large data.

Command: pig or pig -x mapreduce

Also available: Tez mode (pig -x tez) and Spark mode (pig -x spark). This is the 'Pigs live anywhere' motto in action.

Analogy: local mode = practising in your home kitchen; MapReduce mode = cooking in a restaurant kitchen.

HDFS COMMANDS

HDFS commands you will use with Pig

Command	Purpose	Linux cousin
`hdfs dfs -ls /data`	List files in a directory	ls
`hdfs dfs -mkdir -p /data/in`	Create a directory (with parents)	mkdir -p
`hdfs dfs -put f.csv /data/in`	Copy local file into HDFS	cp
`hdfs dfs -copyFromLocal f.csv /data`	Same as -put	cp
`hdfs dfs -get /data/out ./out`	Copy from HDFS to local	cp
`hdfs dfs -copyToLocal /data/f .`	Same as -get	cp
`hdfs dfs -cat /data/out/part-*`	Print file contents	cat
`hdfs dfs -tail /data/f.csv`	Show the end of a file	tail
`hdfs dfs -rm -r /data/out`	Delete a file or directory	rm -r
`hdfs dfs -du -h /data`	Disk usage, human readable	du -h

Inside Grunt: fs -ls /data runs the same HDFS commands without leaving Pig.



Worked example

One dataset, one question, six steps: Which cities bring in the most revenue?

WORKED EXAMPLE

The dataset and Step 0: into HDFS

```
orders.csv
(order_id,customer_id,product,category,qty,price)
1,101,Laptop,Electronics,1,55000
2,102,Mouse,Electronics,3,500
3,103,Notebook,Stationery,10,40
4,101,Pen,Stationery,5,20
5,104,Headphones,Electronics,2,1500
6,102,Chair,Furniture,1,4000
7,105,Desk,Furniture,1,8000
8,103,Laptop,Electronics,1,52000
```

```
customers.csv
(customer_id,name,city)
101,Arun,Chennai
102,Divya,Coimbatore
103,Karthik,Chennai
104,Meena,Madurai
105,Rahul,Coimbatore
```

```
hdfs dfs -mkdir -p /learn/input
hdfs dfs -put orders.csv customers.csv \
    /learn/input/
hdfs dfs -ls /learn/input
```

No header rows: PigStorage does not skip headers. Keep local copies for local mode (Steps 1 to 5).

PIG STEP 1

Load and look: LOAD, DESCRIBE, DUMP

```
$ pig -x local

orders = LOAD 'orders.csv' USING PigStorage(',')
   AS (order_id:int, customer_id:int,
       product:chararray, category:chararray,
       qty:int, price:double);

DESCRIBE orders;
DUMP orders;
```

1 **New idea**

orders is a relation (a bag of tuples).

DESCRIBE shows the schema; DUMP prints the data.

LOAD runs nothing on its own. Only DUMP or STORE triggers execution.

Lazy execution: Pig builds a plan first and runs it only when output is requested. Spark works the same way.

PIG STEP 2

Filter rows and compute columns

```
bulk = FILTER orders BY qty >= 2;
DUMP bulk;

with_amt = FOREACH orders GENERATE
    order_id, customer_id, category,
    qty * price AS amount;
DUMP with_amt;
```

2 **New idea**

FILTER picks rows.

FOREACH ... GENERATE picks and computes columns.

with_amt (with the new amount column) feeds every later step.

Builds on: orders from Step 1.

PIG STEP 3

Group and aggregate: GROUP, SUM, COUNT

```
by_cat = GROUP with_amt BY category;
DESCRIBE by_cat;
```

```
by_cat: {group: chararray,
  with_amt: {(order_id:int, customer_id:int,
              category:chararray, amount:double)}}
```

```
cat_rev = FOREACH by_cat GENERATE
    group AS category,
    SUM(with_amt.amount) AS revenue,
    COUNT(with_amt) AS num_orders;
DUMP cat_rev;
```

3 **The key Pig idea**

After GROUP, each row = a group key plus a BAG of all the original tuples for that key. You then aggregate over that inner bag.

category	revenue	num_orders
Electronics	111500.0	4
Furniture	12000.0	2
Stationery	500.0	2

Hadoop link: GROUP is the MapReduce shuffle. Each key reaches a reducer with all its values; Pig shows those values as a bag.

PIG STEP 4

Join a second dataset: JOIN

```
customers = LOAD 'customers.csv' USING PigStorage(',')
    AS (customer_id:int, name:chararray,
        city:chararray);

joined = JOIN with_amt BY customer_id,
              customers BY customer_id;
DESCRIBE joined;

by_city  = GROUP joined BY customers::city;
city_rev = FOREACH by_city GENERATE group AS city,
    SUM(joined.with_amt::amount) AS revenue;
DUMP city_rev;
```

4 **New idea**

After a join, fields get prefixes to avoid clashes: with_amt::amount, customers::city.

Grouping by city repeats the Step 3 pattern.

Builds on: with_amt from Step 2 and the GROUP pattern from Step 3.

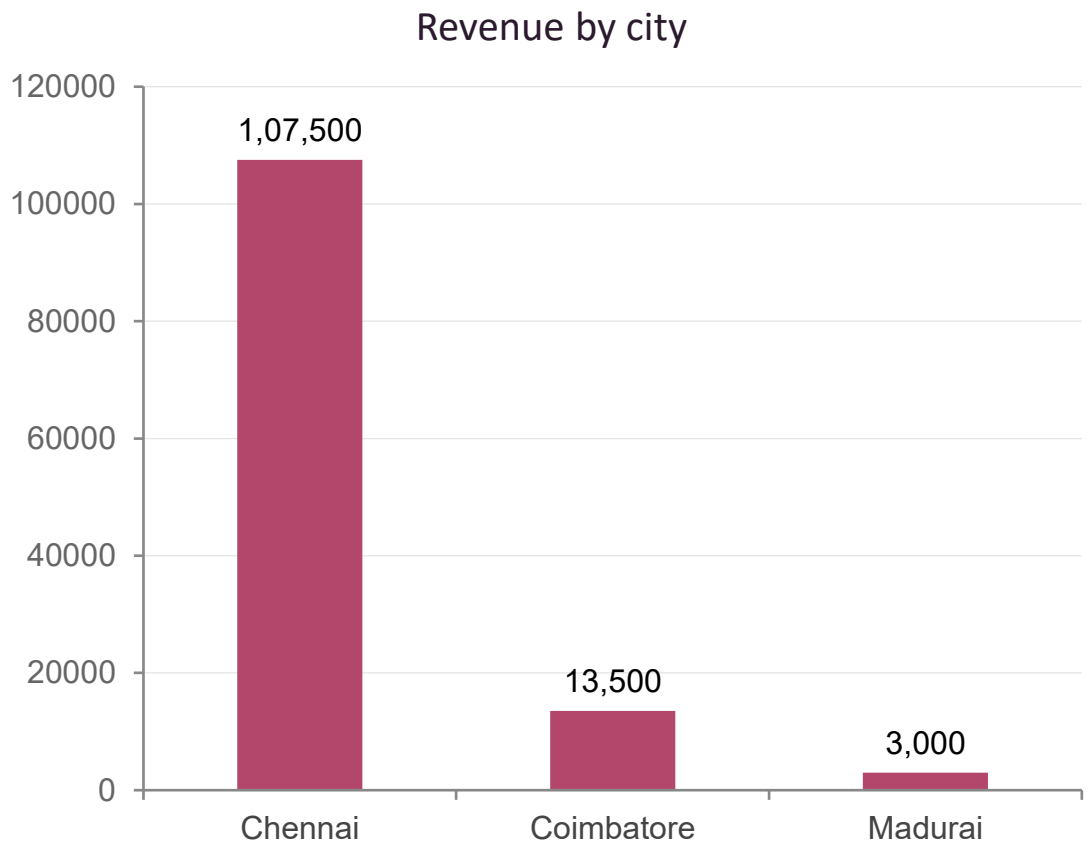
PIG STEP 5

Sort and take the top N: ORDER, LIMIT

```
sorted = ORDER city_rev BY revenue DESC;
top3   = LIMIT sorted 3;
DUMP top3;
```

Result: the question is answered

city	revenue
Chennai	107500.0
Coimbatore	13500.0
Madurai	3000.0



Chennai = Arun (55000 + 100) + Karthik (400 + 52000) = 107500. Let students verify by hand.

PIG STEP 6

Make it real: a script on the cluster

```
-- city_report.pig
orders = LOAD '$INPUT/orders.csv' USING PigStorage(',')
  AS (order_id:int, customer_id:int, product:chararray,
      category:chararray, qty:int, price:double);
customers = LOAD '$INPUT/customers.csv' USING PigStorage(',')
  AS (customer_id:int, name:chararray, city:chararray);
with_amt = FOREACH orders GENERATE customer_id,
             qty * price AS amount;
joined   = JOIN with_amt BY customer_id,
               customers BY customer_id;
by_city  = GROUP joined BY customers::city;
city_rev = FOREACH by_city GENERATE group AS city,
             SUM(joined.with_amt::amount) AS revenue;
sorted   = ORDER city_rev BY revenue DESC;
top3     = LIMIT sorted 3;
STORE top3 INTO '$OUTPUT' USING PigStorage(',');
```

```
$ pig -x mapreduce \
  -param INPUT=/learn/input \
  -param OUTPUT=/learn/output/pig_city \
  city_report.pig

$ hdfs dfs -cat \
  /learn/output/pig_city/part-*
```

6 **What changed**

Paths became $parameters, DUMP became STORE, mode became mapreduce.

Run EXPLAIN top3; and watch the YARN UI: several MapReduce jobs, with results written to HDFS between them. Spark removes that disk traffic.

RECAP

Summary: Pig in one slide

1 What

High-level data-flow platform on Hadoop; Pig Latin compiles to MapReduce.

2 Anatomy

Pig Latin + runtime (Grunt, parser, optimizer, compiler, execution engine).

3 Philosophy

Eats anything, lives anywhere, domestic animal, flies.

4 Data types

Scalars plus tuple, bag, map. A relation is an outer bag.

5 Running

Grunt, script, embedded. Modes: local and MapReduce (also Tez, Spark).

6 Workflow

LOAD → FILTER/FOREACH → GROUP → JOIN → ORDER/LIMIT → STORE.